Capítulo 16: Reúso de Software

SOMMERVILLE, I. Engenharia de software. 9 ed. tradução Ivan Bosnic e Kalinka G.de O. Gonçalves ; revisão técnica Kechi Hirama. São Paulo : Pearson Prentice Hall, 2011. (Disponível na Biblioteca da UFGD)

Capítulo 16: Reúso de software

A engenharia de software baseada em reúso é uma estratégia da engenharia de software em que o processo de desenvolvimento é orientado para o reúso de softwares existentes. Apesar de o reúso ter sido proposto como uma estratégia de desenvolvimento há mais de 40 anos (McILROY, 1968), só em 2000 o 'desenvolvimento com reúso' se tornou a norma para novos sistemas de negócios.

A mudança para o desenvolvimento baseado em reúso foi uma resposta às

exigências de menores custos de produção e manutenção de software, entregas mais rápidas de sistemas e softwares de maior qualidade.

Isso pode dar-se na forma de bibliotecas de programas ou aplicações inteiras.

A engenharia de software baseada em reúso é uma abordagem de desenvolvimento que tenta maximizar o reúso de softwares existentes. As unidades de software reusadas podem ser de tamanhos radicalmente diferentes. Por exemplo:

1. Reúso de sistema de aplicação
2. Reúso de componentes.
3. Reúso de objetos e funções.

Uma vantagem óbvia do reúso de software é a redução dos custos globais de desenvolvimento. Menos componentes de software precisam ser especificados, concebidos, implementados e validados. No entanto, a redução de custos é apenas uma das vantagens do reúso.

1 - COTS
2 - Pronto-completo
3 – Pronto-adaptação
4 – Desenvolvimento

Tabela 16.1 Benefícios do reúso de software

Benefício	Explicação
• Confiança aumentada	Os softwares reusados, experimentados e testados em sistemas em funcionamento provavelmente são mais confiáveis do que um novo software. Seus defeitos de projeto e implementação devem ser encontrados e corrigidos.
• Risco de processo reduzido	O custo do software existente já é conhecido, considerando que os custos de desenvolvimento são sempre uma questão que envolve julgamento. Esse é um importante fator para o gerenciamento de projeto, pois reduz a margem de erro de estimativa de custos de projeto, o que é particularmente verdadeiro quando componentes de software relativamente grandes, como subsistemas, são reusados.
• Uso eficaz de especialistas	Em vez de repetir o mesmo trabalho, especialistas em aplicações podem desenvolver softwares reusáveis que encapsulem seu conhecimento.
• Conformidade com padrões	Alguns padrões, como os de interface de usuário, podem ser implementados como um conjunto de componentes reusáveis. Por exemplo, se os menus em uma interface de usuário forem implementados usando componentes reusáveis, todas as aplicações apresentarão os mesmos formatos de menu para os usuários. O uso de interfaces de usuário-padrão melhora a confiança, pois os usuários cometem menos erros quando são apresentados a interfaces familiares.
• Desenvolvimento acelerado	Muitas vezes, os custos gerais de desenvolvimento não são tão importantes quanto entregar um sistema ao mercado o mais rápido possível. O reúso de um software pode acelerar a produção do sistema, pois pode reduzir o tempo de desenvolvimento e validação.

Tabela 16.2 Problemas com reúso

Problema	Explicação
Maiores custos de manutenção	Se o código-fonte de um sistema ou componentes de software reusáveis não estiverem disponíveis, os custos de manutenção podem ser maiores, pois os elementos reusados do sistema podem tornar-se cada vez mais incompatíveis com as alterações do sistema.
Falta de ferramentas de suporte	Algumas ferramentas de software não suportam o desenvolvimento com reúso. Pode ser difícil ou impossível integrar essas ferramentas com um sistema de biblioteca de componentes. O processo de software assumido por essas ferramentas pode não considerar o reúso. Isso é particularmente verdadeiro para as ferramentas que oferecem suporte a engenharia de sistemas embutidos, exceto para as ferramentas de desenvolvimento orientado a objetos.
Síndrome de 'não-inventado-aqui'	Alguns engenheiros de software preferem reescrever componentes, pois acreditam poder melhorá-los. Isso tem a ver, parcialmente, com aumentar a confiança e, parcialmente, com o fato de que escrever softwares originais é considerado mais desafiador do que reusar softwares de outras pessoas.
Criação, manutenção e uso de uma biblioteca de componentes	Preencher uma biblioteca de componentes reusáveis e garantir que desenvolvedores de software possam utilizar essa biblioteca podem ser ações caras. Processos de desenvolvimento precisam ser adaptados para garantir que a biblioteca seja usada.
Encontrar, compreender e adaptar os componentes reusáveis	Componentes de software precisam ser descobertos em uma biblioteca, compreendidos e, às vezes, adaptados para trabalhar em um novo ambiente. Os engenheiros precisam estar confiantes de que encontrarão, na biblioteca, um componente, antes de incluírem a pesquisa de componente como parte de seu processo normal de desenvolvimento.

O reúso de software é mais eficaz quando está previsto como parte de um programa de reúso de toda a organização.

Um programa de reúso envolve a criação de ativos reusáveis e a adaptação de processos de desenvolvimento para incorporar esses ativos no novo software.

16.1 O panorama de reuso

O reúso é possível em diferentes níveis, desde funções simples até aplicações completas, e normas para componentes reusáveis facilitam o reúso.

A Figura 16.1 define várias possíveis maneiras de implementação de reúso de software.

Figura 16.1 O panorama de reúso

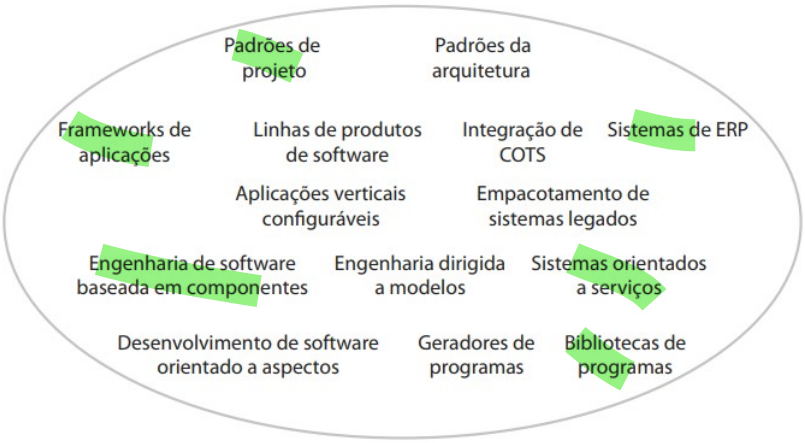


Tabela 16.3 Abordagens que apoiam o reúso de software

Abordagem	Descrição
Padrões de arquitetura	Padrões de arquitetura de software que oferecem suporte a tipos comuns de sistemas de aplicação são usados como base de aplicações. São descritos nos capítulos 6, 13 e 20.
Padrões de projeto	Abstrações genéricas que ocorrem em todas as aplicações são representadas como padrões de projeto, mostrando os objetos abstratos e concretos e as interações. São descritos no Capítulo 7.
Desenvolvimento baseado em componentes	Sistemas são desenvolvidos através da integração de componentes (coleções de objetos) que atendem aos padrões de modelos e componentes. São descritos no Capítulo 17.
Framework de aplicações	Coleções de classes abstratas e concretas são adaptadas e estendidas para criar sistemas de aplicação.
Empacotamento de sistemas legados	Sistemas legados (veja o Capítulo 9) são 'empacotados' pela definição de um conjunto de interfaces e acesso a esses sistemas legados por meio dessas interfaces.
Sistemas orientados a serviços	Sistemas são desenvolvidos pela ligação de serviços compartilhados, que podem ser fornecidos externamente. São descritos no Capítulo 19.
Linhas de produtos de software	Um tipo de aplicação é generalizado em torno de uma arquitetura comum para que esta possa ser adaptada para diferentes clientes.
Reúso de produto COTS	Sistemas são desenvolvidos pela configuração e integração de sistemas de aplicação existentes.
Sistemas de ERP	Sistemas de grande porte que sintetizam a funcionalidade e as regras de negócios genéricos são configurados para uma organização.

Aplicações verticais configuráveis	Sistemas genéricos são projetados para poder ser configurados para as necessidades dos clientes de sistemas específicos.
Bibliotecas de programas	Bibliotecas de classe e funções que implementam abstrações comumente usadas são disponibilizadas para reúso.
Engenharia dirigida a modelos	O software é representado como modelos de domínio e modelos de implementação independentes. O código é gerado a partir desses modelos. São descritos no Capítulo 5.
Geradores de programas	Um sistema gerador incorpora o conhecimento de um tipo de aplicação, e é usado para gerar sistemas nesse domínio a partir de um modelo de sistema fornecido pelo usuário.
Desenvolvimento de software orientado a aspectos	Quando o programa é compilado, os componentes compartilhados são integrados em uma aplicação em diferentes locais. São descritos no Capítulo 21.

.. depende dos requisitos para o sistema em desenvolvimento, a

tecnologia e os ativos reusáveis disponíveis e a expertise da equipe de desenvolvimento. Fatores-chave que devem ser considerados ao planejar o reúso são:

1. O cronograma de desenvolvimento para o software.
2. A expectativa de duração do software.
3. O conhecimento, as habilidades e a experiência do desenvolvimento da equipe
4. A importância do software e seus requisitos não funcionais.
5. O domínio da aplicação.
6. A plataforma em que o sistema será executado.

16.2 Frameworks de aplicações

Os entusiastas do desenvolvimento orientado a objetos sugerem que um dos benefícios principais de se usar uma abordagem orientada a objetos é que eles podem ser reusados em diferentes sistemas.

No entanto, a experiência demonstra que, frequentemente, os objetos são muito pequenos e são especializados para uma aplicação específica. Leva mais tempo compreender e adaptar o objeto do que reimplementá-lo.

Está claro que o reuso orientado a objetos é melhor suportado em um processo de desenvolvimento orientado a objetos por meio das abstrações de alta granularidade, chamadas frameworks.

Como o nome sugere, um framework é uma estrutura genérica estendida para se criar uma aplicação ou subsistema mais específico.

framework como:

“... um conjunto integrado de artefatos de software (como classes, objetos e componentes) que colaboram para fornecer uma arquitetura reusável para uma família de aplicações relacionadas.”

Os frameworks fornecem suporte para recursos genéricos, suscetíveis de serem usados em todas as aplicações de tipos semelhantes.

Os frameworks dão suporte ao reúso de projeto, bem como ao reúso de classes específicas de sistema, pois fornecem uma arquitetura de esqueleto para a aplicação. A arquitetura é definida por classes de objetos e suas interações. As classes são reusadas diretamente e podem ser prorrogadas usando-se recursos, como a herança.

Os frameworks são implementados como uma coleção de classes de objetos concretos e abstratos em uma linguagem de programação orientada a objetos.

Fayad e Schmidt (1997) discutem três classes de frameworks:

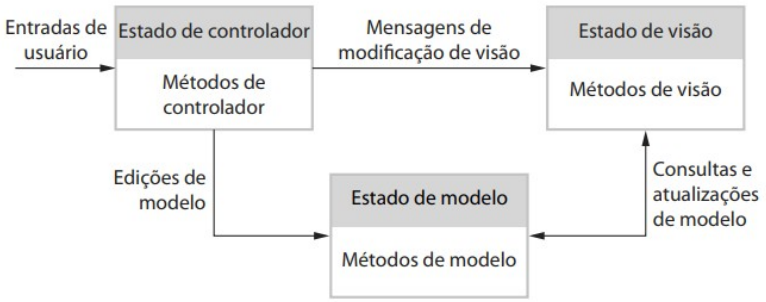
1. Frameworks de infraestrutura de sistema. Esses frameworks apoiam o desenvolvimento de infraestruturas de sistema, como comunicações, interfaces de usuário e compiladores (SCHMIDT, 1997).

2. Frameworks de integração de middleware. Trata-se de um conjunto de normas e classes de objetos associados que oferecem suporte a componentes de comunicação e troca de informações. Exemplos desse tipo de framework incluem o .NET da Microsoft e o Enterprise Java Beans (EJB). Esses frameworks fornecem suporte para modelos de componentes padronizados.

3. Frameworks de aplicações corporativas. Esses estão relacionados com domínios de aplicação específicos, como telecomunicações ou sistemas financeiros (BAUMER et al., 1997). Eles incorporam conhecimentos sobre domínios de aplicações e apoiam o desenvolvimento de aplicações de usuário final

Os frameworks de aplicações Web (WAFs, do inglês web application frameworks) são um tipo de framework mais recente e muito importante. WAFs que apoiam a construção de sites dinâmicos estão amplamente disponíveis. Geralmente, a arquitetura de um WAF é baseada no padrão Composite do MVC (Modelo-Visão-Controlador) (GAMMA et al., 1995).

Figura 16.2 O padrão Modelo-Visão-Controlador



Originalmente, o padrão MVC foi proposto na década de 1980 como uma abordagem de projeto de GUI que permitiu várias apresentações de um objeto e estilos diferentes de interações com cada uma dessas apresentações.

Muitas vezes, os frameworks são implementações de padrões de projeto. Por exemplo, um framework MVC inclui o padrão Observer, o padrão Strategy, o padrão Composite e um número de outros que são discutidos por Gamma et al. (1995).

Os WAFs normalmente incorporam um ou mais frameworks especializados, que oferecem suporte a recursos de aplicações específicos. Embora cada framework inclua uma funcionalidade ligeiramente distinta, a maioria dos WAFs suporta os seguintes recursos:

1. Proteção. WAFs podem incluir classes para ajudar a implementar a autenticação de usuário (login) e controle de acesso, para garantir que os usuários só possam acessar a funcionalidade permitida no sistema.

2. Páginas Web dinâmicas. As classes são fornecidas para ajudar na definição de templates de páginas Web e para preenchê-los dinamicamente, com dados específicos do banco de dados do sistema.

3. Suporte de banco de dados. Geralmente, os frameworks não incluem um banco de dados, mas assumem que um banco de dados separado, como MySQL, será usado. O quadro pode fornecer classes que proporcionam uma interface abstrata para bancos de dados diferentes.

4. Gerenciamento de sessão. Classes para criar e gerenciar sessões (um número de interações com o sistema por um usuário) são geralmente parte de um WAF.

5. Interação de usuário. Atualmente, a maioria dos frameworks Web fornece suporte a AJAX,

que permite que sejam criadas mais páginas Web interativas. Exemplos incluem Angular, React, Vue.js (client-side) e frameworks server-side com suporte integrado, como ASP.NET, Spring MVC e Django com bibliotecas para AJAX.

Para estender um framework, você não altera seu código. Em vez disso, você adiciona a ele classes concretas que herdam operações de classes abstratas.

CHAMO A ATENÇÃO DE VOCÊS.

2011

"Os frameworks são uma abordagem eficaz de reúso, mas são caros para serem introduzidos em processos de desenvolvimento de software. Eles são inerentemente complexos e pode demorar meses para alguém aprender a usá-los. Pode ser difícil e caro avaliar frameworks disponíveis para a escolha do framework mais adequado. A depuração de aplicações baseadas em framework é difícil, pois você pode não compreender como os métodos de framework interagem. Esse é um problema geral dos softwares reusáveis; as ferramentas de depuração podem fornecer informações sobre os componentes de sistema reusados, as quais podem não ser compreendidas por um desenvolvedor."

16.3 Linhas de produto de software

Uma das abordagens mais eficazes para o reúso é criar linhas de produto de software ou famílias de aplicações.

Uma linha de produtos de software é um conjunto de aplicações com uma arquitetura comum e componentes compartilhados, sendo cada aplicação especializada para refletir necessidades diferentes.

O núcleo do sistema é

projetado para ser configurado e adaptado para atender às necessidades de clientes diferentes.

O desenvolvimento de aplicações pela adaptação de uma versão genérica da aplicação significa que uma alta proporção de códigos de aplicação é reusada.

Os testes são simples, pois testes para grande parte da aplicação também podem ser reusados, reduzindo, desse modo, o tempo de desenvolvimento geral da aplicação.

Frameworks de aplicações e linhas de produtos de software têm, obviamente, muito em comum. Ambos oferecem suporte a uma arquitetura e componentes comuns e exigem um novo desenvolvimento para se criar uma versão específica de um sistema. As principais diferenças entre essas abordagens são:

1. Frameworks de aplicação dependem de recursos orientados a objetos como herança e polimorfismo para implementar as extensões para o framework.

-- o código de framework não é modificado, e possíveis modificações limitam-se ao que é permitido pelo framework

-- Linhas de produtos de software não são necessariamente criadas usando-se uma abordagem orientada a objetos. Os componentes de aplicação são alterados, deletados ou rescritos

2. Frameworks de aplicação concentram-se principalmente no apoio técnico e não de domínio específico. Uma linha de produtos de software geralmente incorpora informações detalhadas de domínio e de plataforma.

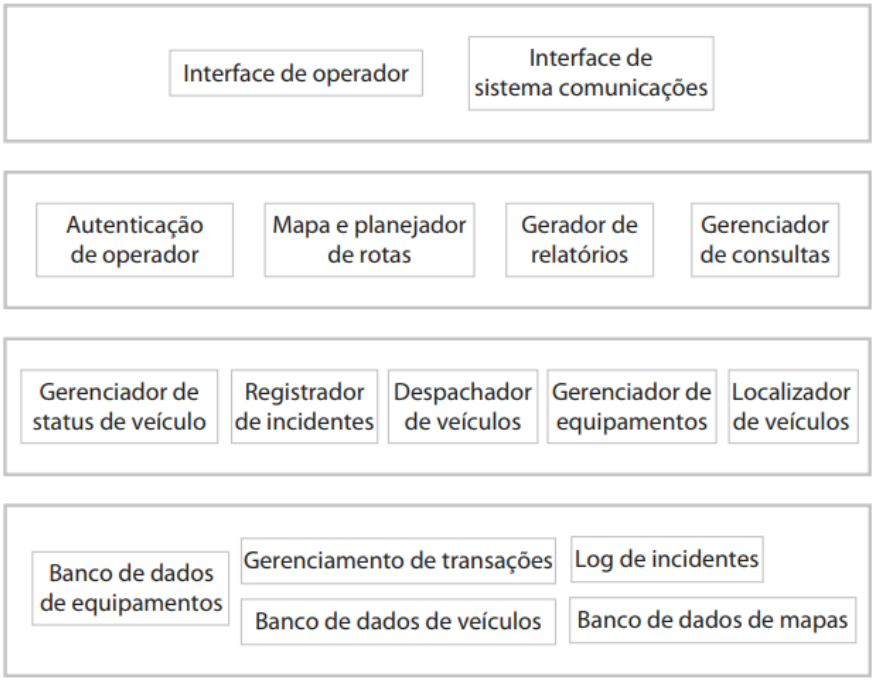
3. Muitas vezes, as linhas de produtos de software são aplicações de controle para o equipamento. Geralmente, os frameworks de aplicação são orientados para software e raramente fornecem suporte para a interface de hardware.

4. Linhas de produtos de software são compostas de uma família de aplicações relacionadas, de propriedade da mesma organização. Quando você cria uma nova aplicação, o ponto de partida frequentemente é o membro mais próximo da família de aplicações, e não a aplicação central genérica.

a arquitetura de uma linha de produtos de software reflete um estilo ou padrão de arquitetura geral de aplicações específicas. Por exemplo, considere um sistema de linha de produto que é projetado para lidar com a expedição de veículos para serviços de emergência.

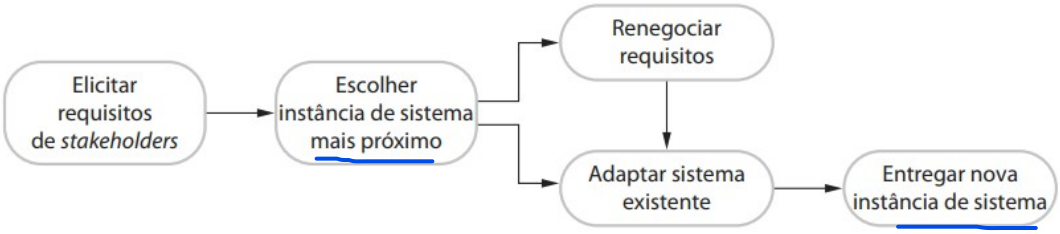
Os desenvolvedores de tal sistema podem negociar versões desse sistema para os serviços de polícia, bombeiros e ambulâncias.

Figura 16.5 A arquitetura de linha de produto de um sistema de despacho de veículos



A Figura 16.6 mostra as etapas necessárias para estender uma linha de produtos de software para se criar uma nova aplicação.

Figura 16.6 Desenvolvimento de instância de produto



As linhas de produtos de software são projetadas para serem reconfiguradas, e essa reconfiguração pode envolver adicionar ou remover componentes do sistema, definir parâmetros e restrições para componentes de sistema e incluir o conhecimento de processos de negócios.

16.4 Reúso de produtos COTS

Um produto de prateleira (COTS, do inglês commercial-off-the-shelf) é um sistema de software que pode ser adaptado às necessidades de diferentes clientes sem alterar o código-fonte do sistema.

Praticamente todos os softwares de desktop e uma grande variedade de produtos de servidor são softwares COTS.

Os produtos COTS são adaptados por mecanismos internos de configuração que permitem que a funcionalidade do sistema seja customizada para se adaptar às necessidades específicas do cliente.

ER

Existem dois tipos de reúso de produtos COTS, chamados sistemas de solução COTS e sistemas integrados COTS.

Sistemas de solução COTS consistem em uma aplicação genérica, de um único fornecedor, configurada para os requisitos do cliente.

Os sistemas integrados COTS envolvem a integração de dois ou mais sistemas COTS (talvez de diferentes fornecedores) para se criar um sistema de aplicação.

16.4.1 Sistemas de solução COTS

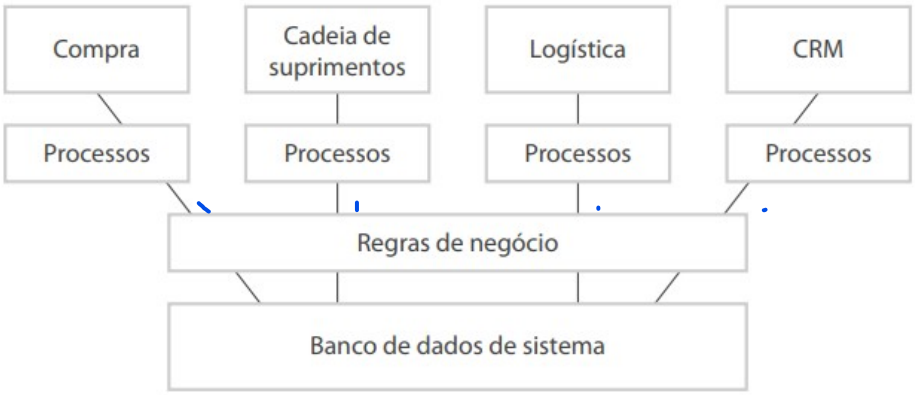
Sistemas de solução COTS são sistemas genéricos de aplicação que podem ser projetados para suportar um tipo de negócio em particular, atividade de negócio ou, por vezes, uma empresa completa.

Por exemplo, um sistema de solução COTS pode ser produzido para dentistas que lidam com consultas, registros dentais, recuperação de pacientes

Em uma escala maior, um sistema de Enterprise Resource Planning (ERP) pode apoiar toda a fabricação, os pedidos e as atividades de gerenciamento de relacionamento com o cliente, em uma grande empresa.

Os sistemas ERP... Para esses sistemas, o processo de configuração envolve a coleta de informações detalhadas sobre negócios do cliente e processos de negócios e a incorporação destes em um banco de dados de configuração, o que exige, muitas vezes, o conhecimento detalhado da configuração, notações e ferramentas, e geralmente é realizado por consultores e clientes.

Figura 16.8 A arquitetura de um sistema ERP



Os sistemas de ERP são usados em quase todas as grandes companhias para oferecer suporte a algumas ou todas as funções.

16.4.2 Sistemas integrados COTS

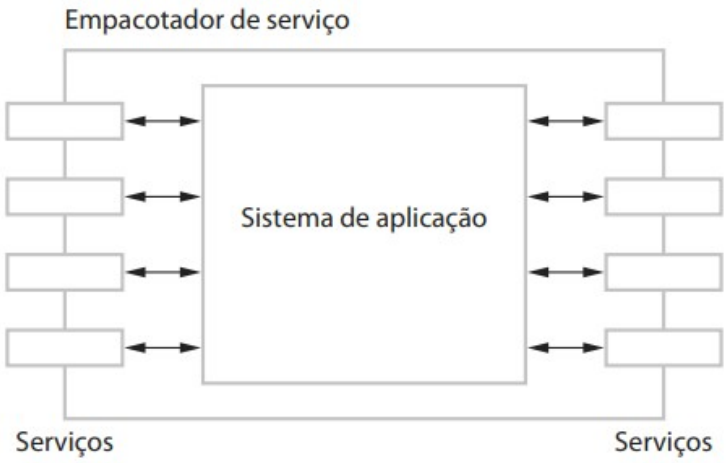
Sistemas integrados COTS são aplicações que incluem dois ou mais produtos COTS ou, às vezes, sistemas de aplicações legados.

Para desenvolver sistemas usando produtos COTS, é necessário fazer algumas opções de projeto:

1. Quais produtos COTS oferecem a funcionalidade mais apropriada?
2. Como os dados serão trocados?
3. Quais características de um produto serão realmente usadas?

A princípio, integrar produtos COTS é o mesmo que integrar quaisquer outros componentes. Você precisa compreender as interfaces do sistema e usá-las exclusivamente para se comunicar com o software. É necessário negociar requisitos específicos por rápido desenvolvimento e reúso; e você precisa projetar uma arquitetura de sistema que permita que os sistemas COTS operem em conjunto

Figura 16.10 Empacotamento de aplicação



Quatro importantes problemas de integração de sistema COTS:

1. Falta de controle de funcionalidade e desempenho
2. Problemas com a interoperabilidade de sistema COTS.
3. Não existe controle sobre a evolução de sistema.
4. Suporte dos fornecedores COTS.

Capítulo 20: Software embutido

SOMMERVILLE, I. Engenharia de software. 9 ed. tradução Ivan Bosnic e Kalinka G.de O. Gonçalves ; revisão técnica Kechi Hirama. São Paulo : Pearson Prentice Hall, 2011. (Disponível na Biblioteca da UFGD)

Capítulo 20: Software embutido

Os computadores são usados para controlar uma vasta gama de sistemas, desde máquinas domésticas simples, controladores de jogos, até plantas inteiras de fabricação. Esses computadores interagem diretamente com dispositivos de hardware.

Seu software deve reagir a eventos gerados pelo hardware e, muitas vezes, emitir sinais de controle em resposta a tais eventos.

Esses sinais resultam em uma ação, como o início de uma chamada de telefone, o movimento de um caractere na tela, a abertura de uma válvula, ou a exibição de status do sistema.

O software é embutido no hardware do sistema, muitas vezes em memória do tipo apenas leitura, e geralmente responde em tempo real a eventos no ambiente do sistema.

Diz-se que o sistema de software tem um deadline para responder a eventos externos em tempo real e, se esse deadline for perdido, o sistema de hardware/software global não funcionará corretamente.

O software embutido é muito importante economicamente porque quase todos os dispositivos elétricos incluem software.

Se você olhar em sua casa, poderá notar que existem três ou quatro computadores pessoais, mas provavelmente você tem 20 ou 30 sistemas embutidos, tais como sistemas de telefones, fogões, micro-ondas etc.

A capacidade de resposta em tempo real é a diferença crítica entre sistemas embutidos e outros sistemas de software, como os sistemas de informações, os sistemas baseados em Web ou os sistemas de software pessoais, cuja principal finalidade é o processamento de dados.

Em um sistema de tempo real, a correção depende tanto da resposta para uma entrada quanto do tempo necessário para gerar essa resposta. Se o sistema demorar muito para responder, a resposta necessária poderá ser ineficaz.

Por exemplo, se um software embutido muito lento controlar um carro com um sistema de frenagem, um acidente poderá ocorrer porque é impossível parar o carro na hora.

Portanto, o tempo é inerente à definição de um sistema de software de tempo real:

"Um sistema de software de tempo real é um sistema cujo funcionamento correto depende tanto dos resultados produzidos pelo sistema quanto do tempo em que esses resultados são produzidos. Um ‘sistema de tempo real’ é um sistema cuja operação é degradada se os resultados não forem produzidos em conformidade com os requisitos de tempo especificados. Se os resultados não forem produzidos de acordo com a especificação de tempo em um ‘sistema de tempo real pesado’, isso é considerado uma falha de sistema"

A resposta no tempo certo é um fator importante em todos os sistemas embutidos, mas nem todos os sistemas embutidos exigem uma resposta muito rápida.

Existem outras diferenças importantes entre sistemas embutidos e outros tipos de sistema de software, além da necessidade de respostas em tempo real:

1. Geralmente, os sistemas embutidos executam continuamente e não param.
2. As interações com o ambiente do sistema são incontroláveis e imprevisíveis.
3. Podem haver limitações físicas que afetem o projeto de um sistema.
4. A interação direta com o hardware pode ser necessária.
5. Questões de segurança e confiabilidade podem dominar o projeto de sistema.

20.1 Projeto de sistemas embutidos

O processo de projeto para sistemas embutidos é um processo de engenharia de sistemas em que os projetistas de software devem considerar em detalhes o projeto e o desempenho do hardware de sistema.

Parte do processo de projeto do sistema pode envolver e decidir quais recursos de sistema serão implementados no software e no hardware.

Os custos e o consumo de energia do hardware são críticos para muitos sistemas de tempo real, embutidos em produtos de consumo, como os sistemas de telefones celulares.

Os processadores específicos projetados para oferecer suporte a sistemas embutidos podem ser usados e, para alguns sistemas, um hardware especial pode ter de ser projetado e construído.

Os sistemas embutidos são sistemas reativos que reagem a eventos em seu ambiente, e a abordagem geral de projeto de software embutido de tempo real é baseada em um modelo de estímulo-resposta. Um estímulo é um evento que ocorre no ambiente do sistema de software que faz com que o sistema reaja de alguma forma. Uma resposta é um sinal ou mensagem enviada pelo software para seu ambiente.

Você pode definir o comportamento de um sistema real listando os estímulos recebidos pelo sistema, as respostas associadas e o tempo em que a resposta deve ser produzida.

Tabela 20.1 Estímulos e respostas para um sistema de alarme contra roubo

Estímulo	Resposta
Único sensor positivo	Iniciar o alarme; acender luzes em torno do local do sensor positivo.
Dois ou mais sensores positivos	Iniciar o alarme; acender luzes em torno dos locais de sensores positivos; chamar a polícia com a localização suspeita do arrombamento.
Queda de tensão entre 10 e 20%	Comutar para bateria reserva; executar o teste de fornecimento de energia.
Queda de tensão de mais de 20%	Comutar para bateria reserva; iniciar o alarme; chamar a polícia; executar o teste de fornecimento de energia.
Falha de fonte de alimentação	Chamar o serviço técnico.
Falha de sensor	Chamar o serviço técnico.
Botão de pânico de console positivo	Iniciar alarme; acender luzes em torno do console; chamar a polícia.
Limpar alarmes	Desligar todos os alarmes ativos; apagar todas as luzes que tiverem sido ligadas.

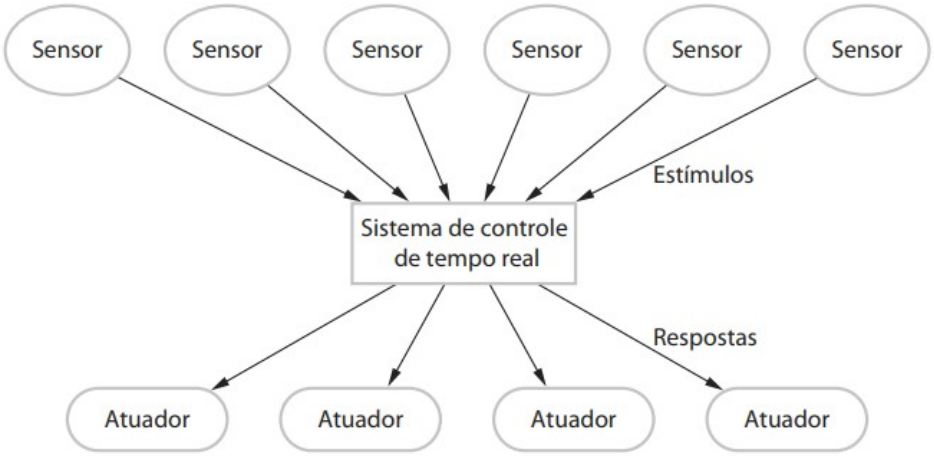
Os estímulos são divididos em duas classes:

1. Periódicos. Estímulos que ocorrem em intervalos previsíveis. Por exemplo, o sistema pode examinar um sensor a cada 50 milissegundos (ms) e agir (responder) em função desse valor de sensor (estímulo).

2. Aperiódicos. Estímulos que ocorrem de forma irregular e imprevisível e geralmente são sinalizados pelo mecanismo de interrupção do computador. Um exemplo desse estímulo seria uma interrupção indicando que uma transferência de E/S foi concluída e que os dados estavam disponíveis em um buffer.

Os estímulos provêm de sensores no ambiente do sistema, e as respostas são enviadas aos atuadores, como mostra a Figura 20.1

Figura 20.1 O modelo geral de sistemas embutidos de tempo real



—----------

Atuadores em sistemas embutidos são componentes responsáveis por transformar sinais de controle provenientes do sistema (geralmente digitais) em ação física, como movimento ou força, interagindo diretamente com o ambiente externo.

Exemplos em Automação Residencial

Motores elétricos acionando persianas e cortinas automáticas conforme sensores de luminosidade detectam variações na luz ambiente.

Atuadores para abrir fechaduras eletrônicas de portas por comando via smartphone ou biometria.

Válvulas eletrônicas em sistemas de irrigação, ativadas automaticamente quando sensores medem baixos níveis de umidade no solo.

Exemplos em Saúde

Bombas de infusão automatizadas com atuadores controlando a liberação de medicamentos de forma precisa conforme parâmetros programados pelo médico.

Atuadores em ventiladores pulmonares, que ajustam a pressão de ar conforme necessidade do paciente, baseando-se em leituras de sensores.

Sistemas portáteis de monitoramento, que aplicam leve pressão para medir sinais vitais por meio de atuadores de pressão.

Exemplos em Indústria e Robótica

Braços robóticos industriais que executam tarefas repetitivas, como montagem, solda ou paletização, utilizando atuadores elétricos e pneumáticos para movimentação.

Válvulas industriais controladas eletronicamente para regular o fluxo de substâncias como gases ou líquidos em processos automatizados, acionadas por atuadores.

Sistemas automotivos autônomos, onde atuadores de direção, frenagem e aceleração recebem comandos do software embarcado para garantir segurança e navegação precisa.

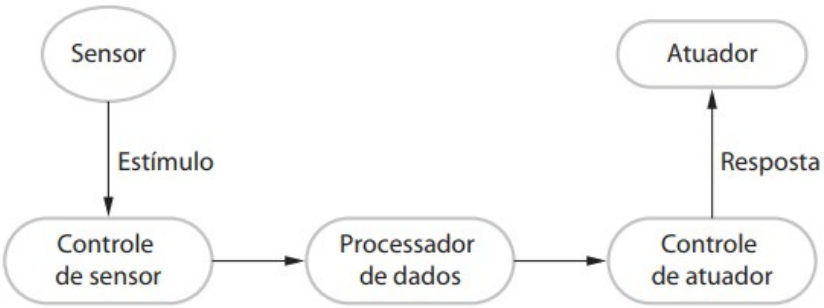
Exemplos em Equipamentos do Cotidiano

Impressoras 3D que utilizam motores de passo (atuadores elétricos) para movimentar o cabeçote e a mesa de impressão de acordo com o projeto digital.

Equipamentos de preparo de café ou chá automáticos, com atuadores que controlam o fluxo de água quente, abertura de compartimentos e movimentos de mistura baseados em comandos do sistema embarcado

—----------

Figura 20.2 Processos de sensores e atuadores



Não há um processo-padrão de projeto de sistemas embutidos.

As seguintes atividades podem ser incluídas em um processo de projeto de software de tempo real:

1. Seleção de plataforma. Nessa atividade, você escolhe uma plataforma de execução para o sistema (ou seja, o hardware e o sistema operacional de tempo real a ser usado). Os fatores que influenciam essas opções incluem as restrições de tempo sobre o sistema, as limitações de potência disponível, a experiência da equipe de desenvolvimento e o preço-alvo para o sistema entregue.

2. Identificação de estímulos/resposta. Isso envolve a identificação dos estímulos que o sistema deve processar e a resposta ou respostas associadas a cada estímulo.

3. Análise de timing. Para cada estímulo e resposta associada, identificam-se as restrições de tempo que se aplicam ao estímulo e ao processamento de resposta. Estes são usados para estabelecer deadlines para os processos do sistema.

4. Projeto de processo. Nesse estágio, agrega-se o estímulo e a transformação da resposta em um número de processos concorrentes. Um bom ponto de partida para projetar a arquitetura de processo são os padrões da arquitetura. Em seguida, otimiza-se a arquitetura de processo para refletir os requisitos específicos que se devem implementar.

5. Projeto de algoritmo. Para cada estímulo e resposta, criam-se algoritmos para efetuar os processamentos necessários. Projetos de algoritmo podem precisar ser desenvolvidos relativamente cedo no processo de projeto para dar uma indicação da quantidade de processamento e tempo necessários para concluir o processamento.

Isso é especialmente importante para tarefas computacionalmente intensivas, como processamento de sinais.

6. Projeto de dados. Você especifica as informações que são trocadas por processos e os eventos que coordenam a troca de informações, e cria estruturas de dados para gerenciar essas trocas de informações. Vários processos concorrentes podem compartilhar essas estruturas de dados.

7. Programação de processo. Você projeta um sistema de programação que garantirá que os processos são iniciados no tempo certo para cumprirem seus deadlines.

A ordem dessas atividades de processo de projeto de software de tempo real depende do tipo de sistema a ser desenvolvido, bem como seus requisitos de processo e plataforma.

⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙

Pesquisa

INPI ou Google Patents

Softwares embutidos.

Verifiquem o processo de desenvolvimento do software.

Mandem no grupo do whatsapp, título do produto e processo utilizado.

⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙

20.1.1 Modelagem de sistema de tempo real

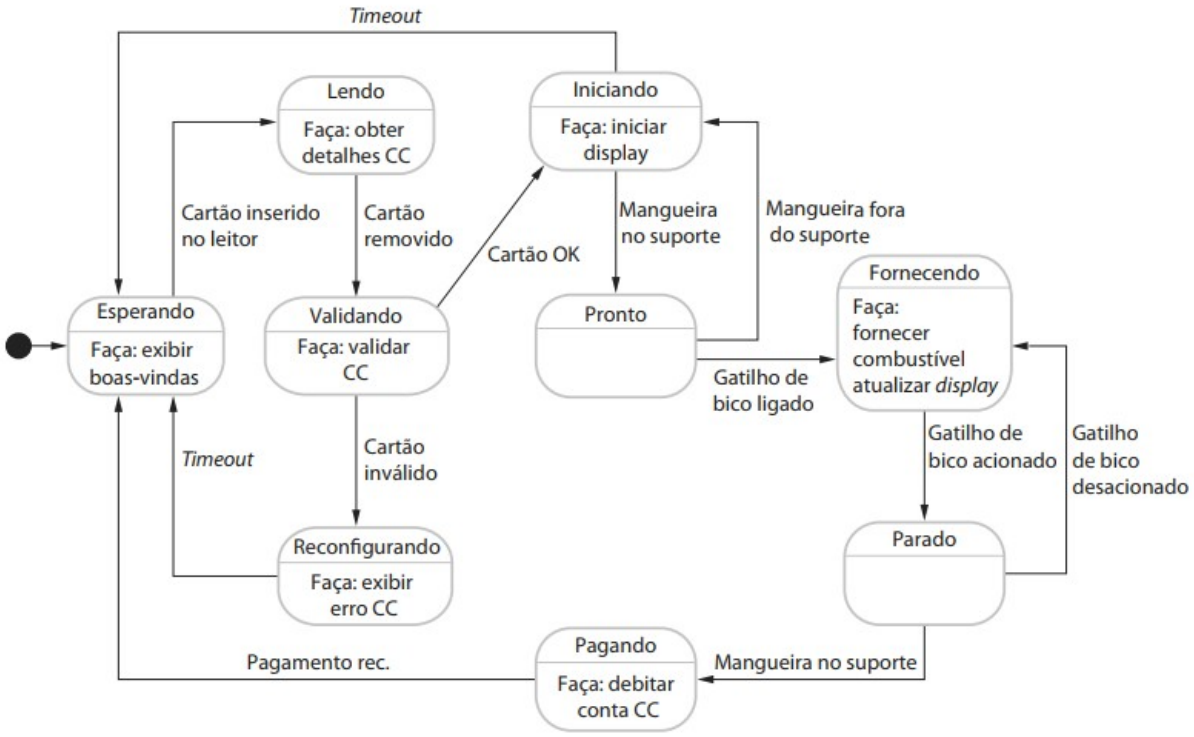
Os eventos aos quais um sistema de tempo real deve reagir costumam fazer com que o sistema se mova de um estado para outro.

Por essa razão, modelos de estado são muitas vezes usados para descrever sistemas de tempo real. Quando um estímulo é recebido, isso pode causar uma transição para um estado diferente.

Os modelos de estado são uma maneira de representar o projeto de um sistema em tempo real, independentemente da linguagem e, portanto, são parte integrante dos métodos de projeto de sistema de tempo real (GOMAA, 1993). A UML suporta o desenvolvimento de modelos de estado baseados em Statecharts (HAREL, 1987; HAREL, 1988). Statecharts são modelos formais de máquina de estado que suportam estados hierárquicos, para que grupos de estados possam ser considerados uma entidade única.

A Figura 20.4 é outro exemplo de um modelo de máquina de estado que mostra o funcionamento de um sistema de software embutido de fornecimento de combustível em uma bomba de gasolina. Os retângulos arredondados representam estados de sistema e as setas representam estímulos que forçam a transição de um estado para outro. Os nomes escolhidos no diagrama de máquina de estado são descritivos. As informações associadas indicam ações tomadas pelos atuadores do sistema ou informações exibidas. Observe que esse sistema nunca se encerra, mas entra em estado de espera quando a bomba não está operando.

Figura 20.4 Modelo de máquina de estado de uma bomba de gasolina



20.1.2 Programação em tempo real

As linguagens de programação para desenvolvimento de sistemas de tempo real precisam incluir recursos para acessar o hardware de sistema, e deve ser possível prever o timing de determinadas operações nessas linguagens.

Os sistemas de tempo real pesados ainda, às vezes, são programados em linguagem assembly para poder atender aos deadlines apertados. São também amplamente usadas linguagens de nível de sistema, como C, que permitem gerar códigos eficientes.

A vantagem de se usar uma linguagem de programação de sistemas como C é que ela permite o desenvolvimento de programas muito eficientes. No entanto, essas linguagens não incluem construções para oferecer suporte à concorrência ou ao gerenciamento de recursos compartilhados.

Porque os sistemas em tempo real devem satisfazer suas restrições de timing, eles podem não ser capazes de usar o desenvolvimento orientado a objetos para sistemas de tempo real pesados.

20.2 Padrões de arquitetura

Os padrões de arquitetura são descrições abstratas e estilizadas de boas práticas de projeto. Eles encapsulam o conhecimento sobre a organização das arquiteturas de sistemas, quando essas arquiteturas devem ser usadas, suas vantagens e desvantagens. Você não deve, no entanto, pensar em um padrão de arquitetura como um projeto genérico para ser instanciado. Em vez disso, use o padrão para entender uma arquitetura e como ponto de partida para criar seu próprio projeto de arquitetura específico.

Como você poderia esperar, as diferenças entre os softwares embutidos e interativos significam que diferentes padrões de arquitetura são usados para sistemas embutidos...

Os padrões de sistemas embutidos são orientados a processos, em vez de orienta dados a objetos e componentes. Nesta seção, discuto três padrões de arquitetura de tempo real comumente usados:

1. Observar e reagir. Esse padrão é usado quando um conjunto de sensores é monitorado e exibido rotineiramente. Quando os sensores mostram que ocorreu algum evento (por exemplo, uma chamada recebida em um telefone celular), o sistema reage, iniciando um processo para tratar esse evento.

2. Controle de ambiente. É usado quando um sistema inclui sensores que fornecem informações sobre o ambiente e atuadores capazes de alterar o ambiente. Em resposta às mudanças ambientais detectadas pelo sensor, sinais de controle são enviados para os atuadores de sistema.

3. Pipeline de processo. Esse padrão é usado quando dados precisam ser transformados de uma representação para outra antes que possam ser processados. A transformação é implementada como uma sequência de etapas de processamento, que podem ser realizadas concorrentemente. Isso permite o processamento de dados muito rápido, porque um núcleo separado ou processador pode executar cada transformação.

20.3 Análise de timing

Uma das atividades importantes no processo de desenvolvimento de software embutido de tempo real é a análise de timing. Em uma análise desse tipo, calcula-se com que frequência cada processo do sistema deve ser executado para garantir que todas as entradas sejam processadas e todas as respostas do sistema sejam produzidas no tempo correto. Os resultados da análise de timing são usados para decidir quão frequentemente cada processo deve executar e como esses processos devem ser programados pelo sistema operacional de tempo real.

A análise de timing para os sistemas de tempo real é particularmente difícil quando os sistemas devem lidar com uma mistura de estímulos periódicos e aperiódicos e repostas. Sendo os estímulos aperiódicos imprevisíveis, é necessário fazer suposições sobre a probabilidade de eles ocorrerem e, portanto, requerem serviços a qualquer momento.

Quando os processadores eram lentos, os estímulos aperiódicos precisavam ser usados para garantir que eventos críticos fossem processados antes de seu deadline, pois, geralmente, os atrasos no processamento envolviam alguma perda para o sistema.

Quando você estiver analisando os requisitos de timing dos sistemas embutidos de tempo real e projetar sistemas para atender a esses requisitos, existem três fatores-chave que você deve considerar:

1. Deadlines. O tempo no qual os estímulos devem ser processados, e o sistema, produzir alguma resposta. Se o sistema não cumprir um deadline, caso seja um sistema de tempo real pesado, essa será uma falha de sistema; em um sistema de tempo real leve, isso indica que o serviço de sistema está degradado.

2. Frequência. O número de vezes por segundo que um processo deve executar o serviço para ter certeza de que o sistema pode cumprir seus deadlines.

3. Tempo de execução. O tempo necessário para processar um estímulo e produzir uma resposta. Muitas vezes, é necessário considerar dois tempos de execução — o tempo médio de execução de um processo e o pior tempo de execução para esse processo.

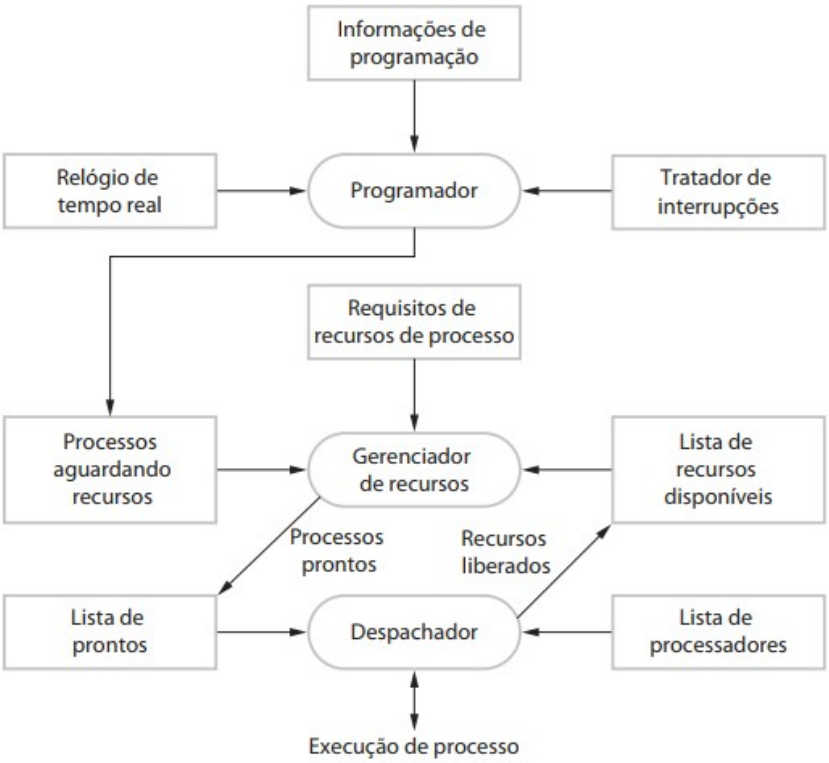
20.4 Sistemas operacionais de tempo real

A plataforma de execução para a maioria dos sistemas de aplicações é um sistema operacional que gerencia recursos compartilhados e fornece recursos como um sistema de arquivos, gerenciamento run-time de processos etc.

Sistemas embutidos muito simples podem ser implementados como sistemas ‘bare metal’ (sem sistema operacional). Os próprios sistemas incluem inicialização e desligamento do sistema, gerenciamento de processo e recursos e programação de processos. Mais comumente, no entanto, os aplicativos embutidos são construídos sobre um sistema operacional de tempo real (RTOS, do inglês real-time operating system), que é um sistema operacional eficiente que oferece os recursos necessários para sistemas em tempo real. Alguns exemplos de RTOS são Windows/CE, Vxworks e RTLinux.

Um sistema operacional de tempo real gerencia os processos e a alocação de recursos para um sistema de tempo real. Ele inicia e para os processos para que os estímulos possam ser tratados e aloca os recursos de memória e processador

Figura 20.12 Componentes de um sistema operacional de tempo real



Os sistemas operacionais de tempo real para grandes sistemas, como sistemas de controle de processos ou de telecomunicações, podem ter recursos adicionais, ou seja, gerenciamento de discos de armazenamento, recursos de gerenciamento de defeitos que detectam e relatam defeitos de sistema e um gerente de configuração que ofereça suporte à reconfiguração dinâmica de aplicações de tempo real.

20.4.1 Gerenciamento de processos

Sistemas em tempo real precisam lidar rapidamente com eventos externos e, em alguns casos, cumprir deadlines para processamento desses eventos. Isso significa que os processos de tratamento de eventos devem ser programados para execução em tempo de detectar tais eventos. Eles também devem ter alocados recursos de processador suficientes para cumprir seu deadline. O gerente de processos em um RTOS é responsável pela escolha dos processos para execução, alocação de recursos de processador e memória, bem como iniciação e parada de execução de processo em um processador.

O gerente de processos precisa gerenciar processos com prioridades diferentes.

o RTOS precisa ser capaz de gerenciar, pelo menos, dois níveis de prioridade para os processos de sistema:

1. Nível de interrupção. É o nível de prioridade mais alto. Ele é alocado para processos que exigem uma resposta muito rápida. Um desses processos será o processo de relógio de tempo real.

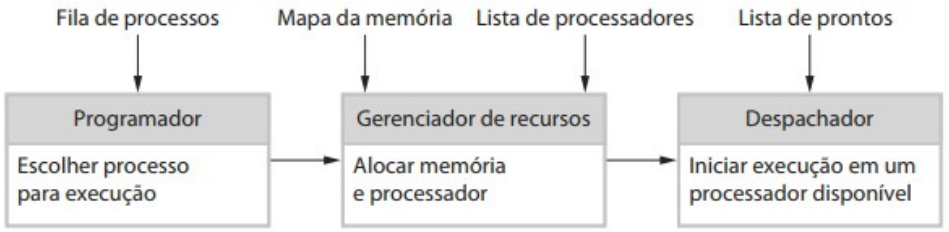
2. Nível de relógio. Esse nível de prioridade é alocado para os processos periódicos

Pode haver outro nível de prioridade alocado para processos em segundo plano (tais como um processo de autoverificação) que não precisam cumprir deadlines de tempo real. Esses processos são programados para execução quando a capacidade de processador está disponível.

Dentro de cada um desses níveis de prioridade, diferentes classes de processo podem ser alocadas para diferentes prioridades.

As ações tomadas pelo sistema operacional para o gerenciamento de processos periódicos são mostradas na Figura 20.13.

Figura 20.13 Ações de RTOS necessárias para iniciar um processo



Às vezes, dois processos com deadlines diferentes devem ser executados no mesmo tique de relógio. Em tal situação, um processo deve ser atrasado. Geralmente, o sistema escolherá retardar o processo com o deadline de mais longo prazo.

A qualquer momento, podem ser executados vários processos, todos com prioridades diferentes. O programador de processos implementa políticas de programação de sistema que determinam a ordem de execução de processos. Existem duas estratégias de programação usadas com frequência:

1. Programação não preemptiva. Uma vez que um processo tenha sido programado para execução, ele é executado até ser concluído ou até que ele seja bloqueado por alguma razão, tal como esperar por uma entrada. No entanto, isso pode causar problemas quando houver processos com prioridades diferentes e um processo de alta prioridade tenha de esperar o fim de um processo de baixa prioridade.

2. Programação preemptiva. A execução de um processo em execução pode ser interrompida caso um processo de prioridade mais alta requisite serviço. Os processos de prioridade mais alta param a execução do processo de prioridade mais baixa e são alocados em um processador.

Dentro dessas estratégias, desenvolveram-se diferentes algoritmos de programação. Estes incluem: programação round-robin, em que um processo é executado de cada vez; programação rate monotonic, na qual há prioridade para o processo com o período mais curto (frequência mais alta); e

programação shortest deadline first, em que é programado o processo na fila com o deadline mais curto (BURNS e WELLINGS, 2009).

Às vezes, dois processos com deadlines diferentes devem ser executados no mesmo tique de relógio. Em tal situação, um processo deve ser atrasado. Geralmente, o sistema escolherá retardar o processo com o deadline de mais longo prazo.

A qualquer momento, podem ser executados vários processos, todos com prioridades diferentes. O programador de processos implementa políticas de programação de sistema que determinam a ordem de execução de processos. Existem duas estratégias de programação usadas com frequência:

1. Programação não preemptiva. Uma vez que um processo tenha sido programado para execução, ele é executado até ser concluído ou até que ele seja bloqueado por alguma razão, tal como esperar por uma entrada. No entanto, isso pode causar problemas quando houver processos com prioridades diferentes e um processo de alta prioridade tenha de esperar o fim de um processo de baixa prioridade.

2. Programação preemptiva. A execução de um processo em execução pode ser interrompida caso um processo de prioridade mais alta requisite serviço. Os processos de prioridade mais alta param a execução do processo de prioridade mais baixa e são alocados em um processador.

Dentro dessas estratégias, desenvolveram-se diferentes algoritmos de programação. Estes incluem: programação round-robin, em que um processo é executado de cada vez; programação rate monotonic, na qual há prioridade para o processo com o período mais curto (frequência mais alta); e programação shortest deadline first, em que é programado o processo na fila com o deadline mais curto (BURNS e WELLINGS, 2009).